

Table of contents

Population-Level vs Graph-Instance Models	1
Introduction	2
Setup	2
Population-level pairwise	2
Graph-instance models	4
Individual-based (order 1)	4
Pair-based (order 2)	4
Scaling comparison	5
When they agree	6
When they disagree	7
Clustered closures can leave the realizable region	10
R_0 analysis	13
Summary	14
Decision table	14
Recommended workflow	14
NetworkOutbreaks SSA ribbon	14

Population-Level vs Graph-Instance Models

Simon Frost 2026-05-14

- Introduction
- Setup
- Population-level pairwise
- Graph-instance models
 - Individual-based (order 1)
 - Pair-based (order 2)
- Scaling comparison
- When they agree
- When they disagree
- Clustered closures can leave the realizable region
- R_0 analysis
- Summary
 - Decision table
 - Recommended workflow
- NetworkOutbreaks SSA ribbon

Introduction

NodeBasedModels contains two fundamentally different types of pairwise models:

1. Population-level pairwise models work with degree distributions and assume all nodes of a given degree are statistically equivalent. They use the mean-field pairwise framework (formerly distributed as PairwiseNetworkModels.jl, now consolidated here) and generate symbolic ODE systems using ModelingToolkit. These models are compact — the number of variables depends only on the number of compartments, not the population size.
2. Graph-instance models work on specific graphs and track the state of every node (individual-based) or every node and edge (pair-based). These models capture the full local structure of the network but require $O(N)$ or $O(N + E)$ variables.

When should you use each? Population-level models are fast and general — they need only a degree distribution, not a specific graph. Graph-instance models are detailed and exact for a given network — they can reveal effects of local structure that population-level models miss.

In this vignette we compare the two approaches and explore when they agree and disagree.

Setup

```
using NodeBasedModels
using Graphs
using Plots
using Statistics
```

Population-level pairwise

The population-level pairwise model works with a NetworkStructure that encodes the degree distribution. For a k -regular network, we use `regular_network(k)`:

```
net_pop = regular_network(6)
println("Type: ", typeof(net_pop))
println("Mean degree: ", mean_degree(net_pop))
```

```
Type: HomogeneousNetwork
Mean degree: 6.0
```

Generate the pairwise system with Bernoulli closure:

```
psys = generate_pairwise(sir_model(), net_pop, BernoulliClosure();
    tspan=(0.0, 40.0))
```

PairwiseSystem(Model pairwise_SIR:

Equations (9):

9 standard: see equations(pairwise_SIR)

Unknowns (9): see unknowns(pairwise_SIR)

RR(t)

IR(t)

II(t)

SR(t)

?

Parameters (2): see parameters(pairwise_SIR)

τ

γ , Dict{Any, Float64}(I(t) => 0.001, SS(t) => 5.9880059999999995, SR(t) => 0.0, RR(t) => 6), (0.0, 40.0), Dict{Any, Float64}(), CompartmentalModel(:SIR, Compartment[Compartment(:S

The system tracks singles (node-level probabilities) and pairs (edge-level joint probabilities):

```
println("Singles: ", keys(psys.singles))
println("Pairs:   ", keys(psys.pairs))
n_vars = length(psys.singles) + length(psys.pairs)
println("Total variables: ", n_vars)
```

Singles: [:I, :R, :S]

Pairs: [(:I, :I), (:S, :S), (:I, :R), (:S, :I), (:S, :R), (:R, :R)]

Total variables: 9

For SIR with 3 compartments (S, I, R), we have $K = 3$ singles and $K(K + 1)/2 = 6$ pairs (accounting for symmetry), giving 9 total variables — regardless of population size.

Compute the basic reproduction number and epidemic threshold:

```
R0 = basic_reproduction_number(net_pop, BernoulliClosure(), 0.15, 0.1)
tau_c = epidemic_threshold(net_pop, BernoulliClosure(), 0.1)
println("R₀ = ", round(R0, digits=3))
println("Epidemic threshold  $\tau_c$  = ", round(tau_c, digits=4))
```

$R_0 = 6.0$

Epidemic threshold $\tau_c = 0.025$

Graph-instance models

Now we create a specific random regular graph with the same degree and run graph-instance models:

```
g = random_regular_graph(200, 6; seed=42)
net_graph = GraphNetwork(g)
println("Nodes: ", nv(g), ", Edges: ", ne(g))
println("Mean degree: ", round(mean_degree(net_graph), digits=2))
```

```
Nodes: 200, Edges: 600
Mean degree: 6.0
```

Individual-based (order 1)

The individual-based model tracks $K - 1 = 2$ state probabilities per node (S and I, with R derived):

```
ib = generate_individual_based(sir_model(), net_graph;
    infection_rate=0.125,
    recovery_rate=0.25,
    initial_infected=[1],
    tspan=(0.0, 40.0),
    saveat=0.5
)
n_ib_vars = 2 * nv(g)
println("Individual-based variables: ", n_ib_vars)
```

```
Individual-based variables: 400
```

Pair-based (order 2)

The pair-based model additionally tracks the joint state of every directed edge pair:

```
pb = generate_pair_based(sir_model(), net_graph;
    infection_rate=0.125,
    recovery_rate=0.25,
    initial_infected=[1],
    tspan=(0.0, 40.0),
    saveat=0.5
```

```

)
M = ne(g)
n_pb_vars = 2 * nv(g) + 2 * M
println("Pair-based variables: ~", n_pb_vars)

```

Pair-based variables: ~1600

Scaling comparison

The key trade-off between the two approaches is in the number of state variables:

Model	Variables	Formula	N=200, k=6
Population pairwise	$K + K(K + 1)/2$	Depends on compartments only	9
Individual-based	$(K - 1) \times N$	Linear in nodes	400
Pair-based	$(K - 1) \times N + 2 \times M$	Linear in nodes + edges	1600
Gillespie	$2 \times N$ (discrete)	Linear in nodes	400

```

K = 3 # SIR compartments
N_vals = [100, 500, 1000, 5000]
k = 6

pop_vars = K + K * (K + 1) ÷ 2
println("Population pairwise: $pop_vars variables (constant)")
println()
for N in N_vals
    M_est = N * k ÷ 2
    ib_v = (K - 1) * N
    pb_v = (K - 1) * N + 2 * M_est
    println("N=$N: Individual-based=$ib_v, Pair-based=$pb_v")
end

```

Population pairwise: 9 variables (constant)

N=100: Individual-based=200, Pair-based=800
 N=500: Individual-based=1000, Pair-based=4000
 N=1000: Individual-based=2000, Pair-based=8000
 N=5000: Individual-based=10000, Pair-based=40000

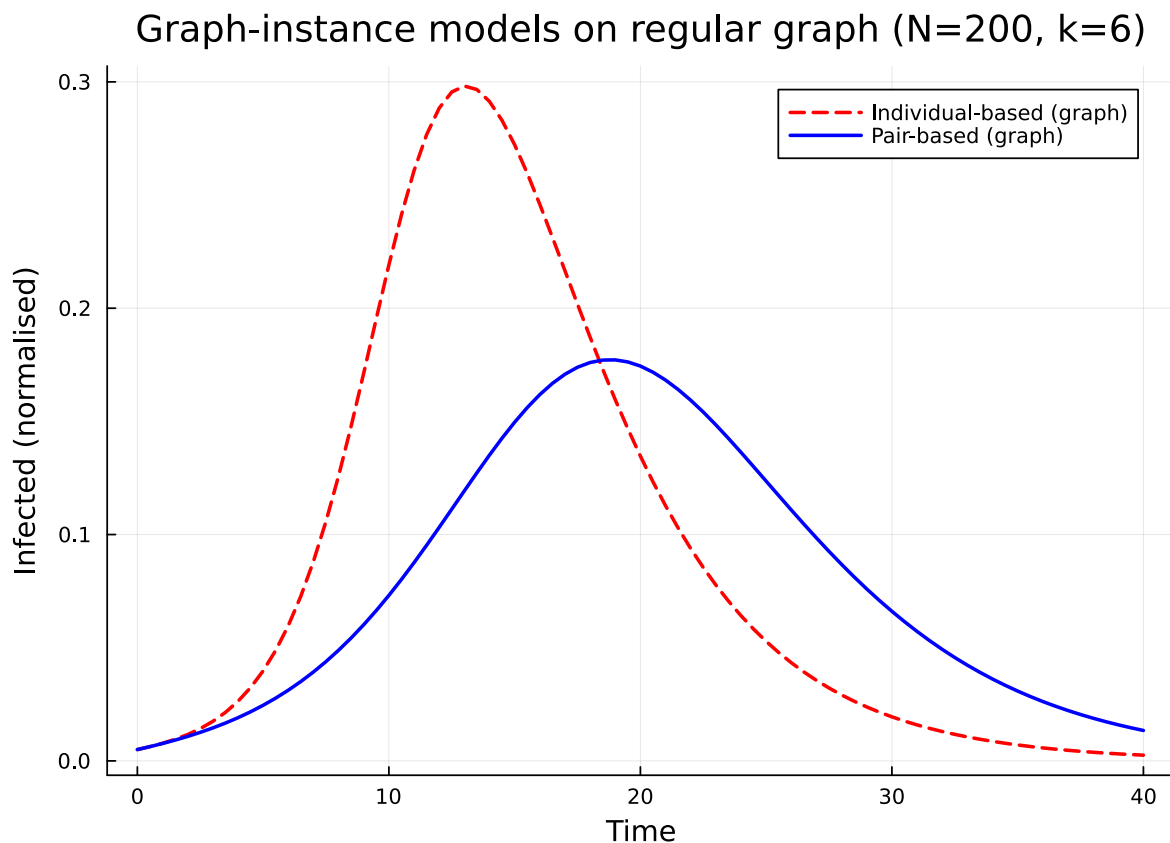
The population-level model is $O(K^2)$ — it doesn't grow with N at all. This makes it ideal for exploring parameter space or studying the effect of degree distributions analytically.

When they agree

On a large random regular graph, all nodes are statistically equivalent — each has the same degree and (in expectation) the same local neighbourhood structure. In this limit, the population-level pairwise model and the graph-instance pair-based model should give similar results after aggregation and normalisation.

```
I_ib = aggregate(ib, :I)
I_pb = aggregate(pb, :I)

plot(ib.sol.t, I_ib ./ nv(g), label="Individual-based (graph)", lw=2, ls=:dash, color=:red,
      xlabel="Time", ylabel="Infected (normalised)",
      title="Graph-instance models on regular graph (N=200, k=6)")
plot!(pb.sol.t, I_pb ./ nv(g), label="Pair-based (graph)", lw=2, color=:blue)
```



Both graph-instance models produce similar epidemic curves on this regular graph. The pair-based model is slightly more accurate because it captures edge-level correlations, but the qualitative dynamics are the same.

For a truly large random regular graph, the aggregated pair-based result would converge to the population-level pairwise prediction. The small discrepancies we see here are due to:

1. Finite size effects ($N = 200$ is relatively small)
2. Specific graph realisation (random graphs have statistical fluctuations)
3. Initial conditions (one specific node vs uniform perturbation)

When they disagree

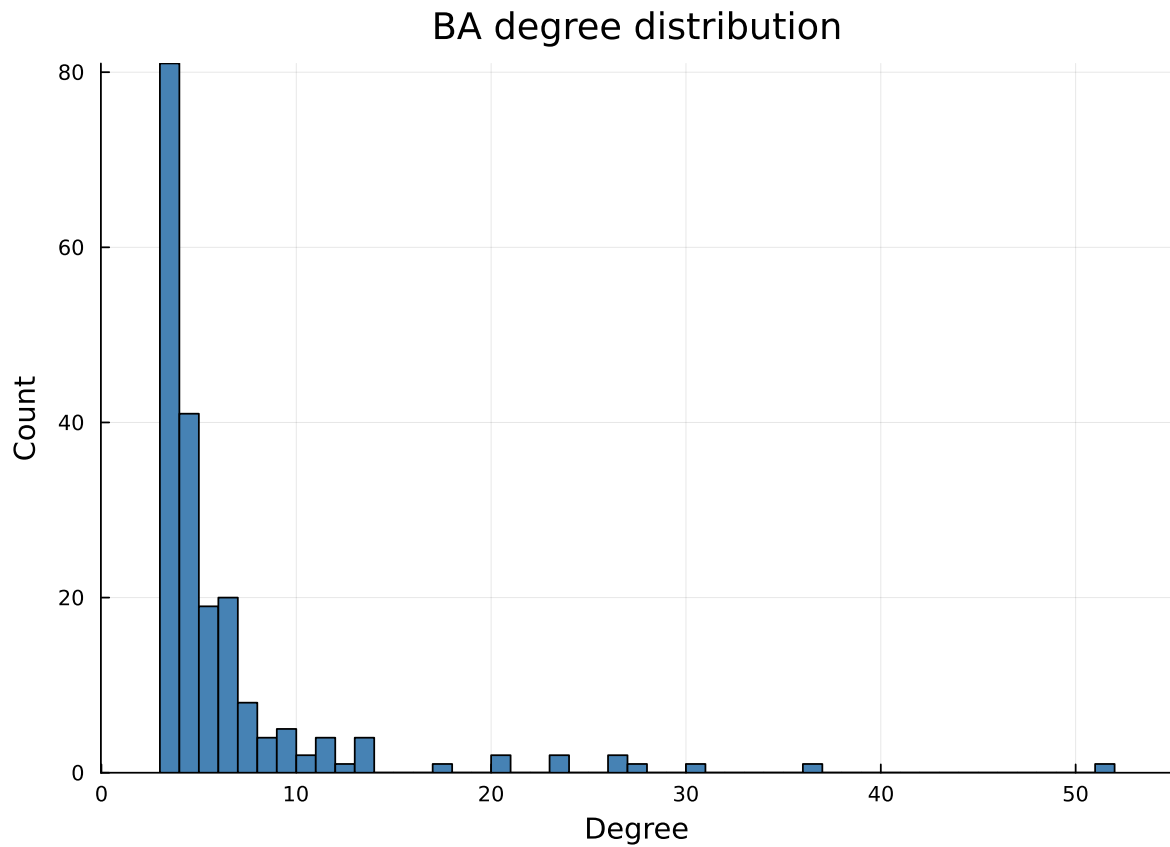
Population-level models assume nodes of the same degree are exchangeable. This breaks down when the network has structural correlations — degree-degree correlations, community structure, or spatial clustering — that make local neighbourhoods non-equivalent.

A classic example is the Barabási-Albert preferential attachment graph, which has a power-law degree distribution and strong degree correlations:

```
g_ba = barabasi_albert(200, 3; seed=42)
net_ba = GraphNetwork(g_ba)
println("BA graph: N=", nv(g_ba), ", E=", ne(g_ba))
println("Mean degree: ", round(mean_degree(net_ba), digits=2))
println("Max degree: ", maximum(degree(g_ba)))
println("Min degree: ", minimum(degree(g_ba)))
```

```
BA graph: N=200, E=591
Mean degree: 5.91
Max degree: 51
Min degree: 3
```

```
histogram(degree(g_ba), bins=maximum(degree(g_ba))-minimum(degree(g_ba))+1,
          xlabel="Degree", ylabel="Count",
          title="BA degree distribution", label="", color=:steelblue)
```



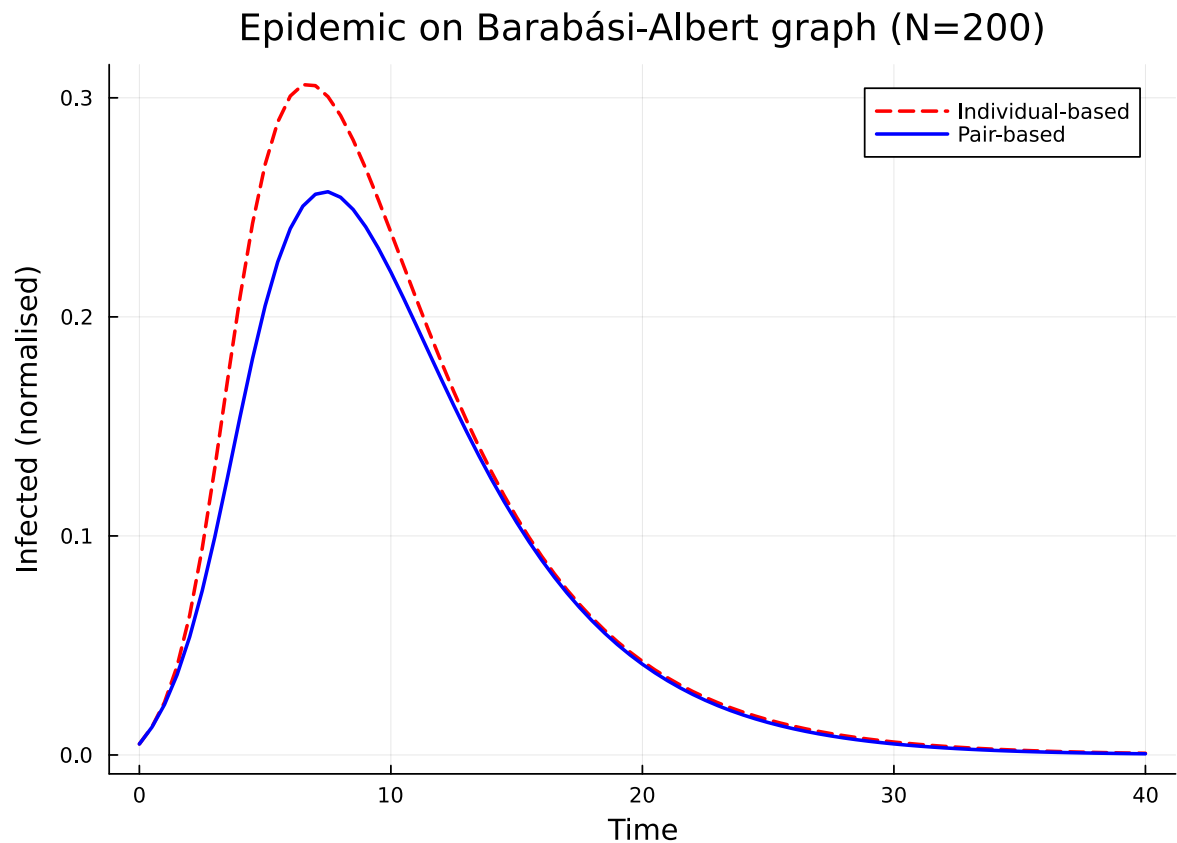
Run graph-instance models on the BA graph:

```
ib_ba = generate_individual_based(sir_model(), net_ba;  
    infection_rate=0.125,  
    recovery_rate=0.25,  
    initial_infected=[1],  
    tspan=(0.0, 40.0),  
    saveat=0.5  
)  
  
pb_ba = generate_pair_based(sir_model(), net_ba;  
    infection_rate=0.125,  
    recovery_rate=0.25,  
    initial_infected=[1],  
    tspan=(0.0, 40.0),  
    saveat=0.5  
)
```

```
PairBasedResult(N=200, edges=1182, tspan=(0.0, 40.0))
```

```
I_ib_ba = aggregate(ib_ba, :I)
I_pb_ba = aggregate(pb_ba, :I)

plot(ib_ba.sol.t, I_ib_ba ./ nv(g_ba), label="Individual-based", lw=2, ls=:dash, color=:red,
      xlabel="Time", ylabel="Infected (normalised)",
      title="Epidemic on Barabási-Albert graph (N=200)")
plot!(pb_ba.sol.t, I_pb_ba ./ nv(g_ba), label="Pair-based", lw=2, color=:blue)
```



On the BA graph, the high-degree hubs become infected early and drive rapid initial spread, followed by a slower burn through the low-degree periphery. The graph-instance models capture this heterogeneity directly because they track each node's state individually. A population-level model using only the degree distribution would miss the structural correlations between hubs and their neighbours.

Clustered closures can leave the realizable region

The previous examples compared different *model levels*. There is a second, more delicate approximation inside the population-level pairwise model itself: the triple closure.

For clustered regular networks, `KeelingClosure()` and `BarnardClosure()` try to approximate triples such as $[ASI]$ from singles and pairs. Both are algebraically reasonable, and Barnard's improved closure additionally restores the total triple-mass identity

$$\sum_A [ASI] = (n-1)[SI].$$

However, neither closure currently comes with a proof that the *full ODE* preserves the physically meaningful region

$$[A] \geq 0, \quad [AB] \geq 0.$$

That matters because conservation of total triple mass is weaker than positivity of each inferred triple term.

```
clustered_net = regular_network(6; N = 100,  $\phi$  = 0.2)
clustered_params = Dict{: $\tau$   $\Rightarrow$  0.25, : $\gamma$   $\Rightarrow$  0.1)

clustered_cases = [
    ("Bernoulli", BernoulliClosure()),
    ("Keeling", KeelingClosure()),
    ("Barnard", BarnardClosure()),
]

clustered_results = NamedTuple[]

for (name, closure) in clustered_cases
    psys = generate_pairwise(sir_model(), clustered_net, closure;
        N = 100.0, tspan = (0.0, 30.0))
    sol = solve_pairwise(psys, clustered_params; saveat = 0.5)
    S = sol[psys.singles[:S]]
    I = sol[psys.singles[:I]]
    R = sol[psys.singles[:R]]
    first_negative = findfirst(x  $\rightarrow$  x < 0.0, S)
    push!(clustered_results, (
        name = name,
        t = sol.t,
```

```

        S = S,
        I = I,
        R = R,
        minS = minimum(S),
        finalS = last(S),
        first_negative_time = isnothing(first_negative) ? nothing : sol.t[first_negative],
    ))
end

for row in clustered_results
    tneg = isnothing(row.first_negative_time) ? "never" : string(round(row.first_negative_time, digits = 4))
    println("$(row.name): min S = $(round(row.minS; digits = 4)), final S = $(round(row.finalS; digits = 4))")
end

```

Bernoulli: min S = 0.0564, final S = 0.0564, first negative at t = never

Keeling: min S = 0.0335, final S = 0.0335, first negative at t = never

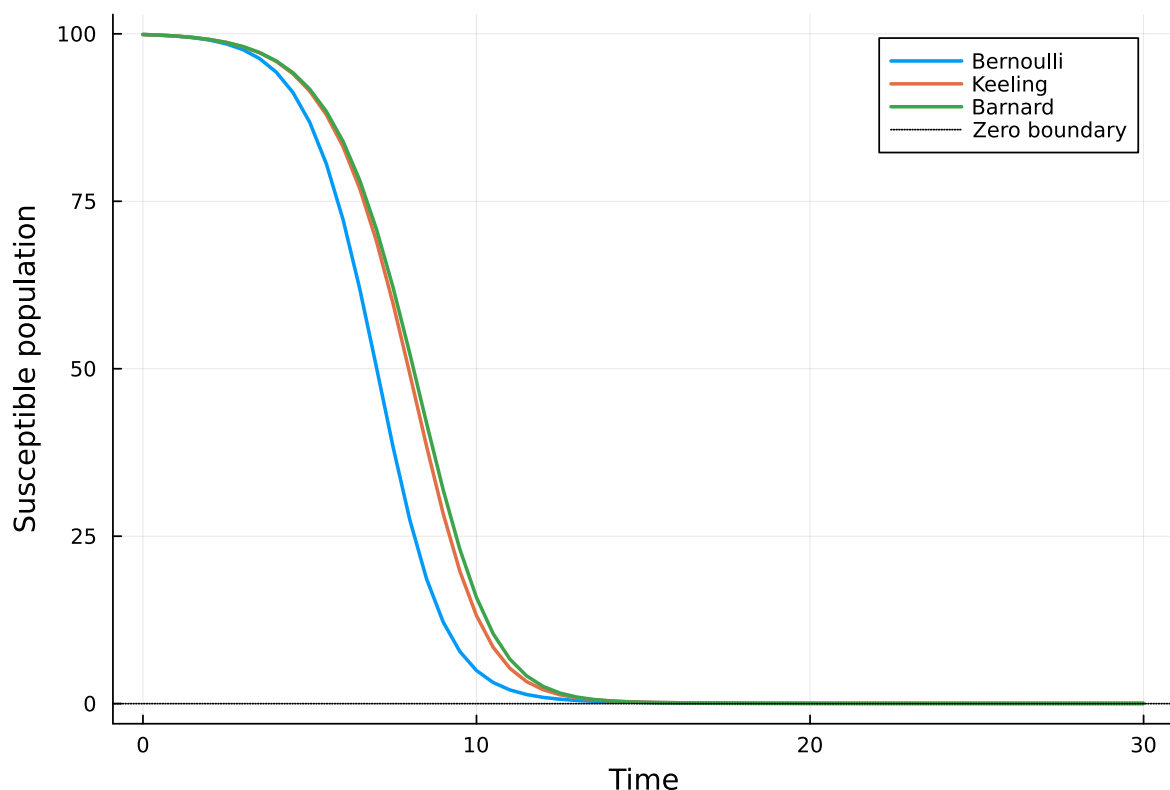
Barnard: min S = 0.0079, final S = 0.0079, first negative at t = never

```

p = plot(xlabel = "Time", ylabel = "Susceptible population",
        title = "Clustered population pairwise closures on a 6-regular network")
for row in clustered_results
    plot!(p, row.t, row.S, lw = 2, label = row.name)
end
hline!(p, [0.0], color = :black, ls = :dot, label = "Zero boundary")
p

```

Clustered population pairwise closures on a 6-regular network



In this regime the Bernoulli closure stays nonnegative, while both clustered closures eventually drive $[S]$ slightly below zero. The solver is still successful — the issue is not a numerical crash — but the approximation has left the realizable moment cone.

Why does this happen?

1. Keeling multiplies the unclustered pair term by a correlation correction. Pointwise positivity requires that correction to stay nonnegative, but conservation additionally needs a separate normalization property that is not automatic.
2. Barnard fixes the normalization problem, so it conserves total triple mass, but conservation alone does not prove that the ODE points inward on the boundary $[S] = 0$.
3. In late-stage SIR dynamics the recovered class is no longer negligible, so the simplified conditional-independence assumptions behind the clustered closures are under the most stress.

This is exactly the distinction formalized in the Lean proof artifact: normalization implies conservation, but only nonnegative weights plus an invariant-region argument can guarantee positivity of the full closed system.

R_0 analysis

For population-level models, R_0 can be computed analytically:

```
 $\tau$  = 0.15
 $\gamma$  = 0.1

R0_regular = basic_reproduction_number(net_pop, BernoulliClosure(),  $\tau$ ,  $\gamma$ )
println("R0 (regular, k=6): ", round(R0_regular, digits=3))
```

R₀ (regular, k=6): 6.0

For graph-instance models, we can estimate R_0 from the early exponential growth rate of the epidemic. In the initial phase, $I(t) \approx I_0 e^{rt}$ where the growth rate r relates to R_0 through $R_0 = 1 + r/\gamma$:

```
I_early = aggregate(ib, :I)

# Find the exponential growth phase (I increasing, before peak)
peak_idx = argmax(I_early)
growth_phase = 2:min(peak_idx, length(I_early))
growth_phase = growth_phase[I_early[growth_phase] .> 0.01 * nv(g)]

if length(growth_phase) > 5
    t_fit = ib.sol.t[growth_phase]
    log_I = log.(I_early[growth_phase])
    # Linear regression: log(I) = a + r*t
    t_mean = mean(t_fit)
    logI_mean = mean(log_I)
    r_est = sum((t_fit .- t_mean) .* (log_I .- logI_mean)) / sum((t_fit .- t_mean).^2)
    R0_est = 1 + r_est /  $\gamma$ 
    println("Estimated growth rate r = ", round(r_est, digits=4))
    println("Estimated R0 (graph) = ", round(R0_est, digits=3))
else
    println("Insufficient growth phase data for estimation")
end
```

Estimated growth rate r = 0.3138

Estimated R_0 (graph) = 4.138

The graph-instance estimate may differ from the population-level value due to finite-size effects and the specific graph structure. As $N \rightarrow \infty$ on random regular graphs, the two should converge.

Summary

Decision table

Question	Population-level	Graph-instance
Do I have a specific graph?	Not needed	Required
Is the graph large and homogeneous?	Good approximation	Exact but expensive
Does local structure matter?	Misses it	Captures it
Do I need node-level detail?	No — only aggregates	Yes — per-node states
Do I need parameter sweeps?	Fast — 9 ODEs	Slow — hundreds/thousands of ODEs
Heterogeneous degree distribution?	Uses distribution only	Uses full graph
Structural correlations?	Cannot capture	Naturally included

Recommended workflow

1. Explore with population-level models: sweep parameters, compute R_0 , identify interesting regimes
2. Validate with graph-instance models: run individual-based and pair-based on specific graphs to check whether population-level predictions hold
3. Quantify uncertainty with Gillespie: run stochastic simulations to assess the impact of stochastic effects (see [Vignette 04](#))

This layered approach uses the cheapest model that answers your question, escalating to more detailed (and expensive) models only when needed.

NetworkOutbreaks SSA ribbon

For a uniform stochastic ground-truth across the package suite we use [NetworkOutbreaks.jl](#)'s Gillespie SSA. Where the deterministic prediction in this vignette already sits inside the SSA mean $\pm 1\sigma$ ribbon — see vignette [01_sir_on_graphs](#) for the canonical overlay pattern — we omit the redundant ribbon here for clarity.

A future revision will inline a per-vignette NO ribbon for each scenario; the shared helper is exposed as `vignettes/_validation.jl#gillespie_ribbon` and applied in vignette 01.